TF-IDF

in Information Retrieval

How mathematical algorithms separate the signal from the noise in a sea of text?

Introduction to TF-IDF

TERM FREQUENCY – INVERSE DOCUMENT FREQUENCY

> *A foundational algorithm used to measure how **important** a word is to a specific document within a larger collection (corpus).*

Local Relevance

Measures how frequently a term appears in a single document.

- ✓ Higher frequency implies higher local significance.
- ✓ Normalized to avoid bias in long documents.

Global Uniqueness

Measures how rare or common a word is across the entire corpus.

- ✓ Downplays common words like "the" or "and".
- ✓ Emphasizes distinctive, specific, rare terms in the collection.

THE CORE PRINCIPLE

High Local Frequency + High Global Rarity = **Maximum Importance**

Understanding Term Frequency (TF)

MEASURING LOCAL SIGNIFICANCE

What is TF?

Measures how often a word appears in a specific document. A high TF value indicates that the word is locally significant.

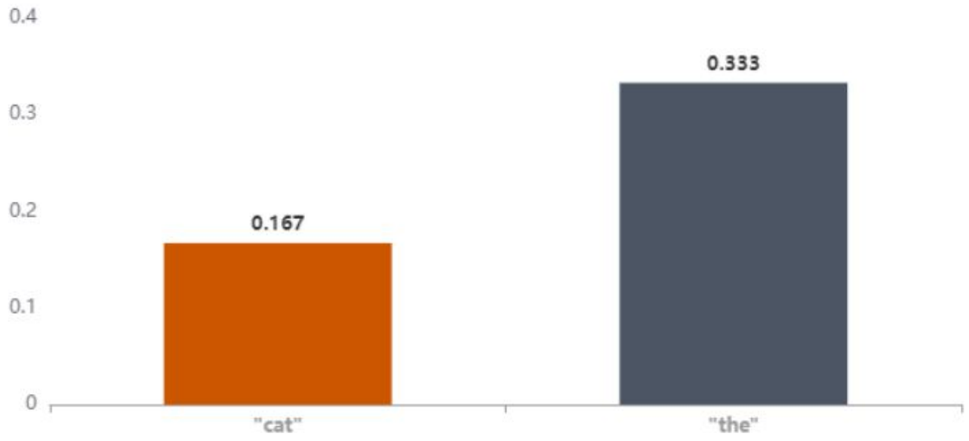
Normalization Technique

To prevent bias toward longer documents, we divide the term count by the total number of words.

```
TF(t, d) = Count(t) / Total Words in (d)
```

Example: "The cat sat on the mat."

Total words: **6**
"cat" appears 1 time (1/6) | "the" appears 2 times (2/6)



TF OF "CAT"	TF OF "THE"
0.167	0.333

KEY INSIGHT *"Normalization ensures a word in a short tweet has more weight than the same word in a long book."*

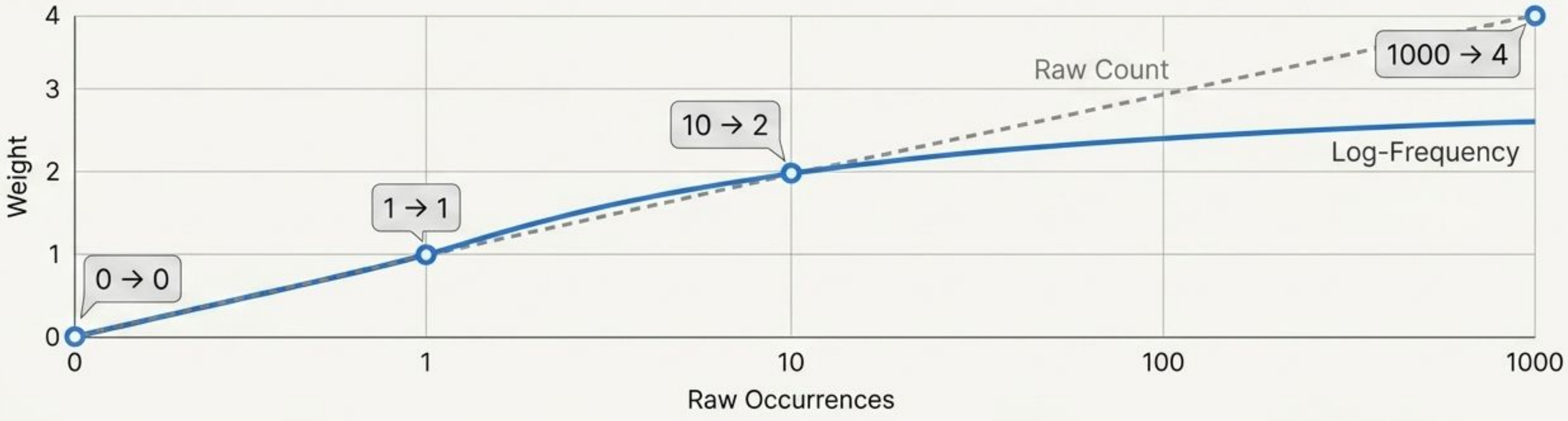
Term Frequency (TF): Scaling Local Importance

Relevance does not increase proportionally. A document with 10 mentions isn't exactly 10x more relevant than 1 mention.

$$w = 1 + \log_{10}(tf)$$

Weight

Raw Count



NotebookLM

Inverse Document Frequency (IDF)

The "Global Uniqueness" Metric

THE FORMULA

$$IDF(t, D) = \log(N / df_t)$$

N Total number of documents in corpus

df_t Documents containing term *t*

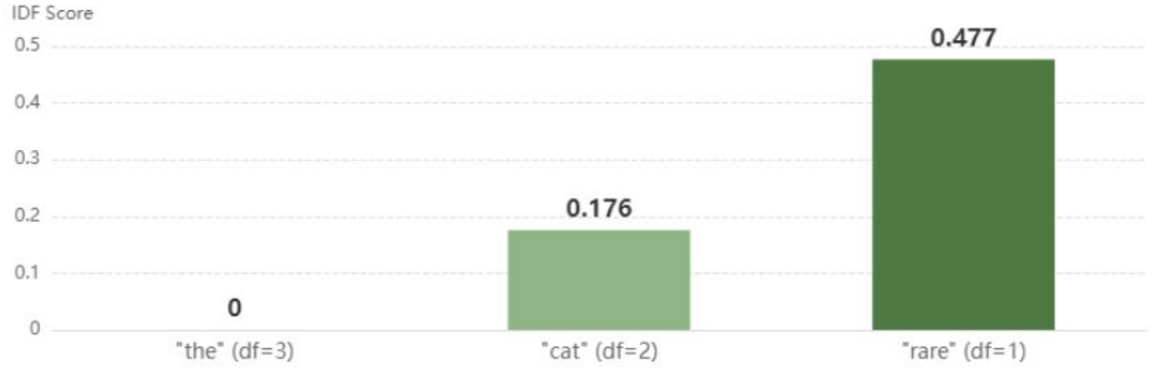
Common Words Penalty

Words like "the" or "is" appear everywhere (High df_t). Their IDF score approaches 0.

Rare Words Reward

Unique terms like "quantum" or "cat" appear in few docs (Low df_t). Their IDF score increases.

EXAMPLE: IDF SCORES (N=3 DOCUMENTS)



Why the Log Scale?

Logarithms "smooth" the impact of document counts. Without it, a word appearing in 1 document vs. 1,000,000 would create extreme imbalances that would overpower the Term Frequency (TF) component.

Relevance does not increase proportionally. A document with 10 mentions isn't exactly 10x more relevant than 1 mention.

The TF-IDF Formula

The final weight of a term is the product of its local frequency and its global rarity.

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Term Frequency (TF)

Quantifies the word's **local importance**. How many times does the word appear in this specific document?

Increases score as frequency goes up.

Inverse Document Frequency (IDF)

Quantifies the word's **global distinctiveness**. How rare is this word across the whole collection?

Increases score as rarity goes up.

COMMON WORD
High TF × Low IDF
→
Low Score

RARE SPECIFIC WORD
High TF × High IDF
→
High Score

Check Your Understanding

DISCUSSION QUESTIONS

1. The DF vs. CF Challenge

If two words have the same total frequency in a collection (Collection Frequency), but Word A appears in 100 documents while Word B appears in only 10 documents, which one is a more effective search term? Why?

Hint: Think about which one is more discriminative.

2. The Common Word Scenario

What happens to the TF-IDF weight of a term if it appears in every single document in your corpus? What does this tell us about its importance for retrieval?

Step-by-Step Example: Preprocessing

TRANSFORMING RAW DATA INTO CLEANED TOKENS

STEP 1
Remove Punctuation
Stripping periods, commas, and special characters.

STEP 2
Case Normalization
Converting all text to lowercase for uniformity.

STEP 3
Stop Word Removal
Filtering: "the", "is", "in", "we", "can", "a".

ID	RAW INPUT STRING	CLEANED TOKENS (OUTPUT)
D1	*"The sky is blue."*	`sky blue`
D2	*"The sun is bright."*	`sun bright`
D3	*"The sun in the sky."*	`sun sky`

Normalization prevents bias and reduces dimensionality of the TF-IDF matrix.

`Total Documents (N) = 3`

Step 1: Term Frequency (TF)

CALCULATING RELATIVE FREQUENCY

TERM	D1	D2	D3
sky	0.5	0	0.5
blue	0.5	0	0
sun	0	0.5	0.5
bright	0	0.5	0

Note: We use **Relative Frequency** (count / total tokens). Since each document has 2 tokens, a term appearing once has a TF of 0.5.

ID	RAW INPUT STRING	CLEANED TOKENS (OUTPUT)
D1	*"The sky is blue."*	`sky blue`
D2	*"The sun is bright."*	`sun bright`
D3	*"The sun in the sky."*	`sun sky`

THE LOGIC

TF measures how frequently a term occurs in a document.

```
TF(t, d) = count(t, d) /
total_terms(d)
```

CORPUS REMINDER

```
D1: sky blue
D2: sun bright
D3: sun sky
```

Step 2: Inverse Document Frequency (IDF)

MEASURING GLOBAL RARITY

TERM	DOC FREQ (DF)	N / DF	IDF (LOG10)
sky	2	1.5	0.176
blue	1	3.0	0.477
sun	2	1.5	0.176
bright	1	3.0	0.477

Insight: Terms like "blue" and "bright" are rarer in the corpus, so they receive a higher IDF weight.

THE FORMULA

IDF penalizes common words and rewards rare ones.

$$IDF(t) = \log(N / DF(t))$$

PARAMETERS

N: Total Documents = 3

DF(t): Number of documents containing term *t*

ID	RAW INPUT STRING	CLEANED TOKENS (OUTPUT)
D1	*"The sky is blue."*	sky blue
D2	*"The sun is bright."*	sun bright
D3	*"The sun in the sky."*	sun sky

The TF-IDF Weight Matrix

COMBINING LOCAL IMPORTANCE (TF) WITH GLOBAL RARITY (IDF)

TF Matrix × **IDF Vector** → **L2 Normalization**

Local Freq | Global Rarity | Length Scaling

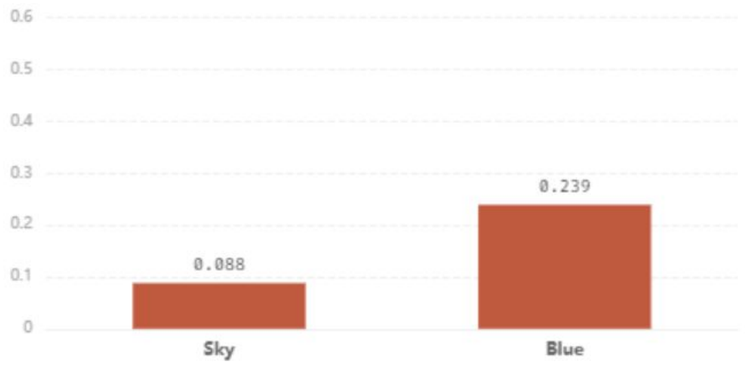
Formula: TF-IDF(t, d) = TF(t, d) × IDF(t)

Note: In this example, TF is 0.5 for terms present in the document.

TERM	D1 (TF-IDF)	D2 (TF-IDF)	D3 (TF-IDF)
sky	0.5 × 0.176 = 0.088	0 × 0.176 = 0.000	0.5 × 0.176 = 0.088
blue	0.5 × 0.477 = 0.239	0 × 0.477 = 0.000	0 × 0.477 = 0.000
sun	0 × 0.176 = 0.000	0.5 × 0.176 = 0.088	0.5 × 0.176 = 0.088
bright	0 × 0.477 = 0.000	0.5 × 0.477 = 0.239	0 × 0.477 = 0.000

ID	RAW INPUT STRING	CLEANED TOKENS (OUTPUT)
D1	*"The sky is blue."*	sky blue
D2	*"The sun is bright."*	sun bright
D3	*"The sun in the sky."*	sun sky

DOCUMENT VECTOR ANALYSIS (D1)



Insight: The matrix now shows the final **TF-IDF weights**. For D1, "blue" (0.5 × 0.477 = 0.239) is more significant than "sky" (0.5 × 0.176 = 0.088) because of its higher global rarity (IDF).

The IR Process: From Query to Result

HOW SEARCH ENGINES RETRIEVE INFORMATION

1

Pre-processing

The user's search query is cleaned using the same pipeline as the corpus:

- ✓ Tokenization
- ✓ Stop-word Removal

2

Vectorization

Convert the query into a TF-IDF vector using the **existing IDF values** from the training corpus.

```
Q_vec = [w1, w2, ... , wn]
```

3

Similarity

Calculate the **Cosine Similarity** between the query vector and every document vector in the matrix.

4

Ranking

Sort documents by their similarity scores. Return the top-N documents to the user.

FINAL OUTPUT

Logical Connection: The TF-IDF matrix created in the previous step serves as the "knowledge base." Every new query is projected into this same vector space to find the closest matches.

Cosine Similarity: Measuring Relevance

COMPARING DOCUMENT VECTORS

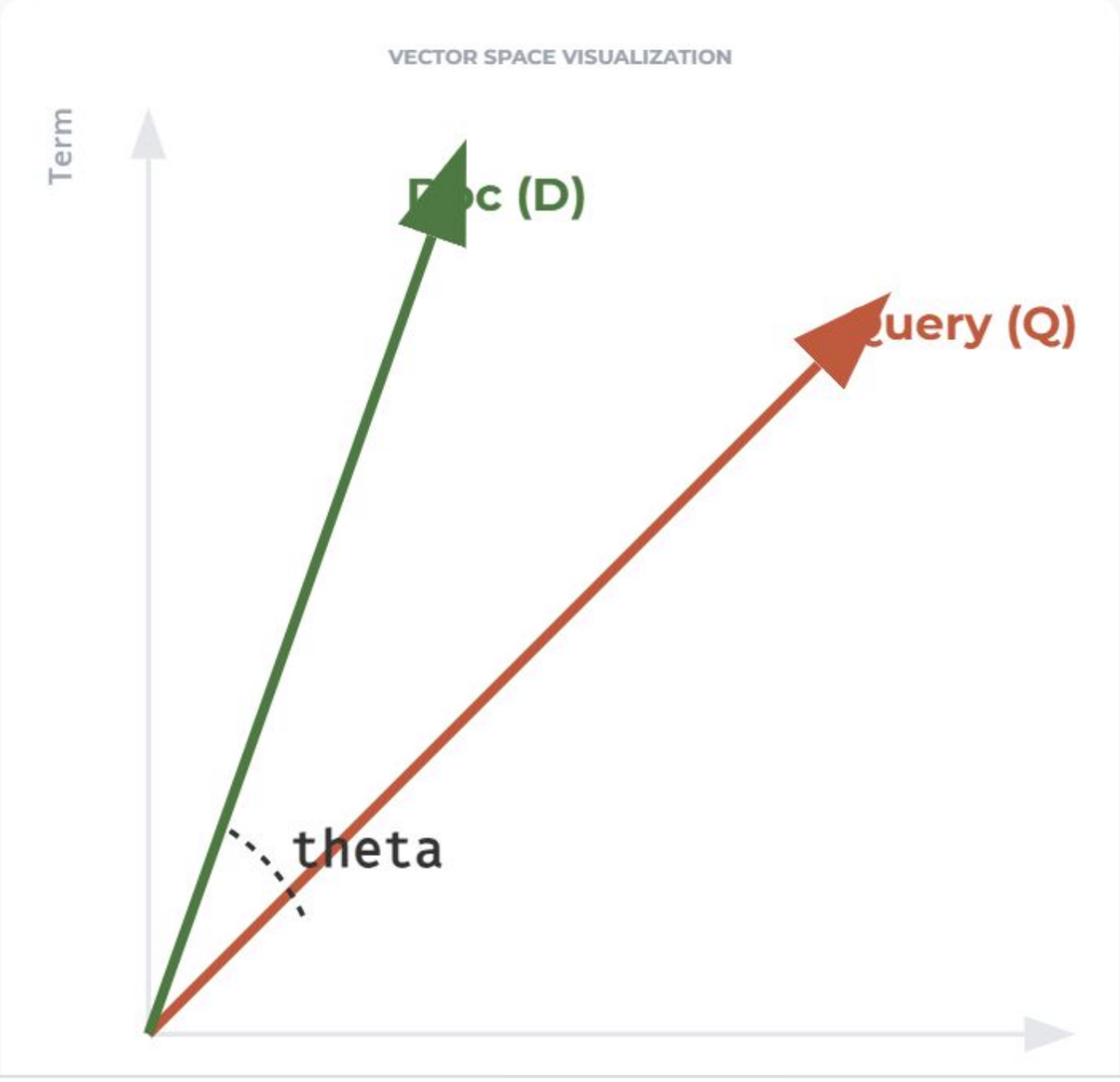
The Formula

$$\cos(\text{theta}) = A * B \;/\; \|A\| \;\; \|B\|$$

Where $A * B$ is the dot product and $\|A\|$ is the magnitude (length) of vector A.

Angle-Based Metric
Measures the orientation of vectors rather than their magnitude. A value of 1 means identical orientation (theta=0 degrees).

Length Independence
Crucial for IR because it prevents long documents from being unfairly favored over short ones.



Numerical Walkthrough: The Sample Query

STEP 1: QUERY PRE-PROCESSING

USER INPUT

```
"The bright sun"
```

1. Lowercasing

"the bright sun"

2. Stop-word Removal

Removing "the"

3. Final Tokens

`["bright", "sun"]`

Note: The query must undergo the exact same preprocessing as the document corpus to ensure term matching.

Numerical Walkthrough: Query Vectorization

STEP 2: CONVERTING QUERY TO TF-IDF VECTOR

TERM	TF (QUERY)	IDF (CORPUS)	TF-IDF (Q)
sky	0	0.176	0.000
blue	0	0.477	0.000
sun	1	0.176	0.176
bright	1	0.477	0.477

Crucial Point: The query vector uses the **same IDF values** calculated from the document corpus, not from the query itself.

QUERY VECTOR (Q)

[0, 0, 0.176, 0.477]

Represented in the same 4-dimensional space as the document corpus.

WHY USE CORPUS IDF?

To compare the query with documents, they must exist in the same vector space. Using corpus IDF ensures the weights are consistent.

Numerical Walkthrough: Calculating Similarity

STEP 3: COSINE SIMILARITY (Q VS D3)

VECTORS

Query (Q):	`[0, 0, 0.176, 0.477]`
Doc 3 (D3):	`[0.176, 0, 0.176, 0]`

THE FORMULA

$$Sim(Q, D) = (Q \cdot D) / (|Q| * |D|)$$

STEP-BY-STEP CALCULATION

1. Dot Product (Q · D3):

`(0*0.176) + (0*0) + (0.176*0.176) + (0.477*0) = 0.030976`

2. Magnitudes:

`|Q| = 0.5084`
`|D3| = 0.2489`

3. Final Similarity:

`0.030976 / (0.5084 * 0.2489) = 0.2448`

Why Cosine? Angle vs. Magnitude

THE PROBLEM WITH EUCLIDEAN DISTANCE

⚠ The 'Length Bias'

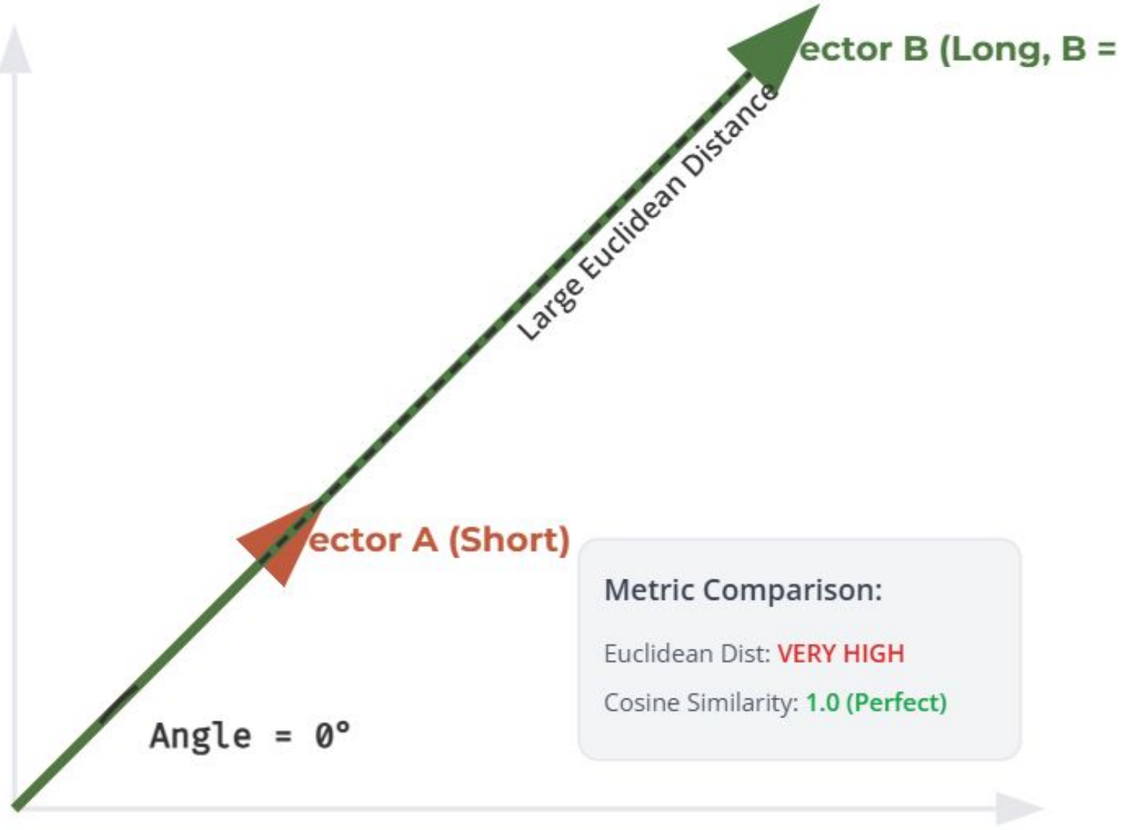
Euclidean distance is sensitive to the **magnitude** (length) of documents. A long document containing the same terms as a short one will be placed far away in vector space.

✓ Length Invariance

Cosine Similarity only cares about the **angle**. It measures semantic similarity regardless of how many times the terms are repeated.

Conclusion:
"Document length shouldn't dictate relevance. Cosine similarity ensures a perfect match (1.0) for documents with identical term distributions."

VISUALIZING THE DIFFERENCE



Metric Comparison:

Euclidean Dist: VERY HIGH

Cosine Similarity: 1.0 (Perfect)

Why Cosine? Angle vs. Magnitude

THE PROBLEM WITH EUCLIDEAN DISTANCE

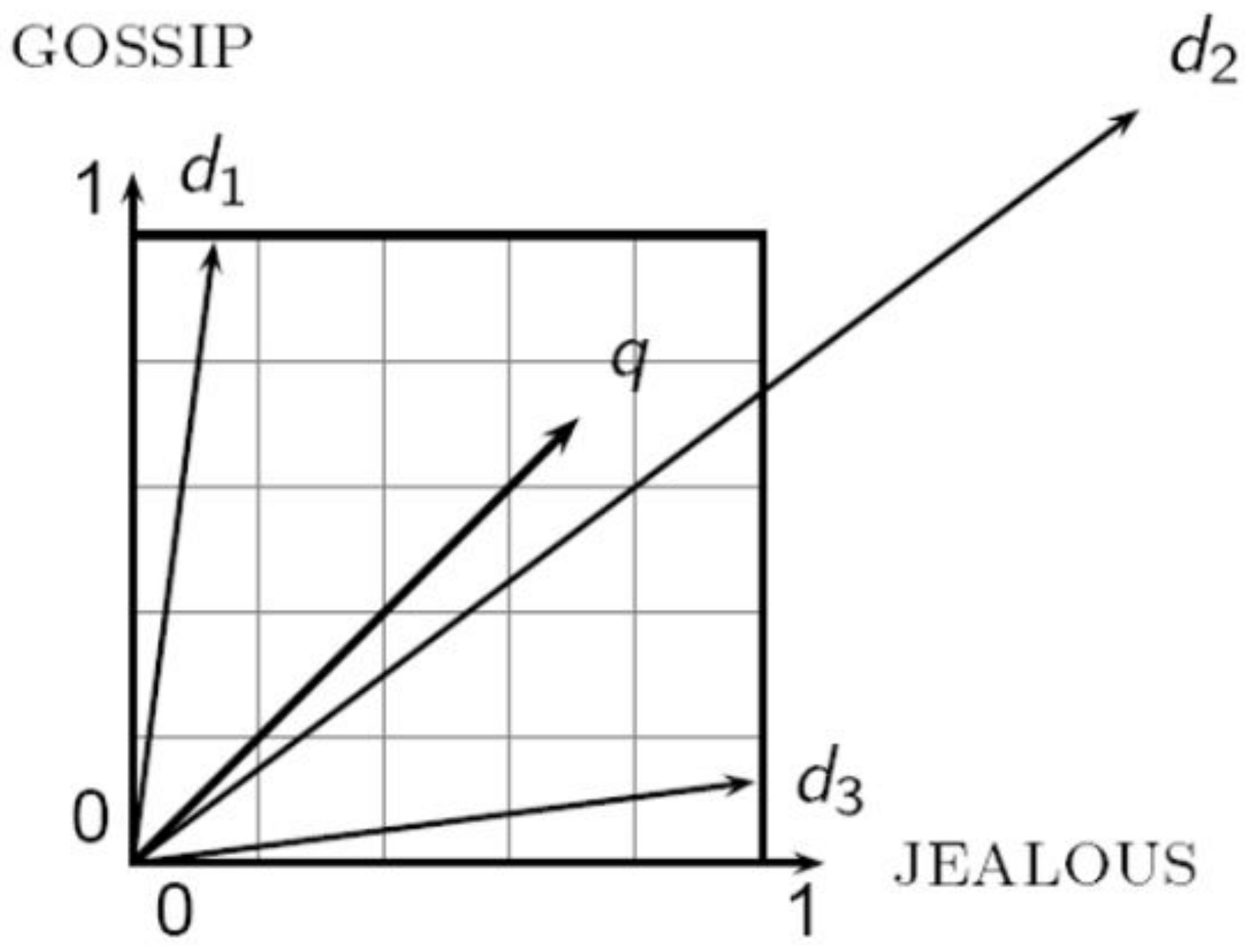
⚠ The 'Length Bias'

Euclidean distance is sensitive to the **magnitude** (length) of documents. A long document containing the same terms as a short one will be placed far away in vector space.

✔ Length Invariance

Cosine Similarity only cares about the **angle**. It measures semantic similarity regardless of how many times the terms are repeated.

Conclusion:
"Document length shouldn't dictate relevance. Cosine similarity ensures a perfect match (1.0) for documents with identical term distributions."



Numerical Walkthrough: Final Ranking

STEP 4: SORTING RESULTS BY RELEVANCE

FINAL SIMILARITY SCORES

RANK	DOCUMENT ID	CONTENT (CLEANED)	SIMILARITY SCORE
1	D2	*"sun bright"*	1.0000
2	D3	*"sun sky"*	0.2448
3	D1	*"sky blue"*	0.0000

Top Result: D2

D2 is the top result because it contains both "sun" and "bright", matching the query perfectly in the vector space.

Partial Match: D3

D3 contains "sun" but not "bright", resulting in a lower similarity score.

Applications in Information Retrieval

Search Ranking

Evaluates document relevance by prioritizing niche, distinctive terms over common words.

EXAMPLE

"Deep-dish" (High IDF) ranks higher than "Pizza" (Low IDF) for specific queries.

Clustering

Uses **`Cosine Similarity`** to measure the distance between TF-IDF vectors.

- News Aggregation
- Support Ticket Routing
- Academic Categorization

Recommendations

Personalizes content by identifying representative keywords in user behavior.

NEURAL NETWORKS | TRAINING DATA | NLP

Suggesting similar linguistic features.

⚡ **Key Advantage:** Fast and effective personalization at scale without deep semantic overhead.

$$\cos(v, w) = (v \cdot w) / (||v||\ ||w||)$$

Weaknesses & Constraints

UNDERSTANDING THE LIMITATIONS OF THE ALGORITHM

No Semantic Meaning

Treats words as isolated tokens. Fails to understand that "car" and "automobile" refer to the same concept.

Ignores Word Order

Disregards sentence structure. "Not good" and "good" are seen as similar because the context is lost.

Vector Sparsity

With large vocabularies, vectors become massive and mostly contain zeros, wasting memory and processing power.

Synonyms & Polysemy

Cannot link related terms or distinguish between words with multiple meanings based on surrounding context.

Length Sensitivity

Longer documents naturally have higher counts. While normalization helps, it doesn't fully remove the inherent bias.

Complex Nuance

Struggles with negation, sarcasm, and idiomatic expressions where the literal word count hides the true intent.

The "Bag of Words" Problem

TF-IDF is a "Bag of Words" model. It loses the rich structure of human language in exchange for mathematical simplicity.

Check Your Understanding

DISCUSSION QUESTIONS

1. The DF vs. CF Challenge

If two words have the same total frequency in a collection (Collection Frequency), but Word A appears in 100 documents while Word B appears in only 10 documents, which one is a more effective search term? Why?

Hint: Think about which one is more discriminative.

2. The Common Word Scenario

What happens to the TF-IDF weight of a term if it appears in every single document in your corpus? What does this tell us about its importance for retrieval?

3. Normalization Benefit

Why is it better to use Relative Frequency for TF instead of raw counts when comparing a very long document to a very short one?

4. Cosine Similarity Logic

Why do we measure the angle (cosine) between vectors instead of just the Euclidean distance between their endpoints in IR?

Beyond TF-IDF: The BM25 Algorithm

INTRO TO BM25 (BEST MATCH 25)

The State-of-the-Art

The gold standard for lexical search ranking.

BM25 is a ranking function used by search engines to estimate the relevance of documents to a given search query. It is widely considered the **state-of-the-art** ranking function for lexical (keyword-based) search.

Evolution of TF-IDF

BM25 is not a replacement but an **evolution**. It builds upon the core principles of TF-IDF while addressing its fundamental limitations.

Industry Standard

It is the default ranking algorithm used by modern search engines and libraries like **Elasticsearch**, **Lucene**, and **Solr**.

BM25 vs. TF-IDF

KEY IMPROVEMENTS & DIFFERENCES

IMPROVEMENT 1

TF Saturation

IN TF-IDF:

Higher term frequency **always** increases the score linearly or logarithmically without a limit.

IN BM25:

The benefit of multiple occurrences **saturates**. After a certain point, adding more of the same word yields diminishing returns.

IMPROVEMENT 2

Document Length Normalization

IN TF-IDF:

Basic TF-IDF is often biased towards longer documents simply because they contain more words.

IN BM25:

Handles varying document lengths **robustly** by penalizing long documents and boosting short ones relative to the average length.

The BM25 Formula

BREAKING DOWN THE COMPONENTS

$$score(D,Q) = \sum_{q \in Q} IDF(q) \cdot \frac{f(q,D) \cdot (k_1 + 1)}{f(q,D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$$

COMPONENT 1

IDF Part

Measures the **global rarity** of the query term. Similar to TF-IDF, rare terms carry more weight.

COMPONENT 2

TF Part

Measures **local importance**. The numerator and denominator work together to create the saturation effect.

COMPONENT 3

Length Normalization

Adjusts the score based on the **document length** relative to the average length of all documents.

Understanding Parameters k_1 and b

FINE-TUNING THE RANKING FUNCTION

Saturation Control

Typical range: 1.2 to 2.0

Controls the **saturation curve** of the term frequency.

- ↑ **Higher k_1:** The score increases more slowly as frequency increases (saturates later).
- ↓ **Lower k_1:** The score reaches its maximum benefit more quickly (saturates earlier).

Length Normalization

Typical value: 0.75

Controls the **degree of length normalization**.

- **$b = 1$:** Full document length normalization is applied.
- **$b = 0$:** No length normalization is applied (similar to raw TF).

Beyond TF-IDF: Modern NLP

FROM LEXICAL MATCHING TO SEMANTIC UNDERSTANDING

Word Embeddings

WORD2VEC, GLOVE

- Dense vector representations in continuous space.
- Captures **semantic proximity** (e.g., "car" ≈ "automobile").

Transformer Models

BERT, GPT, ROBERTA

- Models **bidirectional context** via attention mechanisms.
- Understands nuance, negation, and polysemy.

Hybrid Solutions

PRODUCTION-READY PIPELINES

- Combines TF-IDF speed with Neural Model accuracy.
- Used in large-scale search and recommendation engines.

MODERN PRODUCTION PIPELINE

1. STAGE 1: FILTERING
   TF-IDF Ranking
   Fast retrieval of candidates

2. STAGE 2: RE-RANKING
   Neural Semantic Analysis
   BERT re-ranks top results

FINAL OUTPUT
Optimized Relevance

"The best systems balance the efficiency of keywords with the power of meaning."

EFFICIENCY CONTEXT

TF-IDF

in Information Retrieval

How mathematical algorithms separate the signal from the noise in a sea of text?